

Spatially Aware Opponent Modeling in 3D Multi-Agent Reinforcement Learning

Mohammad Abdullah
23i-0121
FAST-NUCES

Ahsan Azad
23i-0113
FAST-NUCES

Muhammad Hasaan
23i-0094
FAST-NUCES

Abstract—Opponent modeling in multi-agent reinforcement learning has been studied almost exclusively in 2D environments. We present a tag-based predator-prey setting in Project Malmö where two predators learn to tag a co-learning prey using MAPPO extended with an opponent modeling (OM) head trained jointly with the policy via cross-entropy loss over predicted prey actions. Agent observations are encoded through a 3D convolutional voxel encoder over a $5 \times 5 \times 5$ block visibility window. Distributed rollout collection across $N=2$ parallel Malmö workers achieves a $1.61 \times$ training speedup over single-environment training. Against a simultaneously learning prey our method achieves a cumulative win rate of 0.51, outperforming adapted implementations of MARLeOM [1] and OMIS [2]. Both baselines require structural changes to operate in a 3D continuous-observation setting.

I. INTRODUCTION

Single-agent RL has produced strong results on games and control tasks, but most practical deployment scenarios involve multiple interacting agents. This introduces *non-stationarity*: the environment appears non-stationary to any single agent because other agents are also updating their policies. Opponent Modeling (OM) addresses this by explicitly representing the policies of other agents so each agent can anticipate their behavior.

Existing OM work has primarily targeted 2D environments. Even complex benchmarks like StarCraft II [3] operate on a flat map. We extend OM to a Minecraft 3D predator-prey arena, where agents must reason about spatial structure in three dimensions. A voxel-based observation encoder feeds a jointly-trained OM head inside MAPPO, and rollout collection is parallelized across two independent Malmö instances.

Prior Minecraft RL work relied on imitation learning from human demonstrations [4]. Our setup is reward-driven and targets adaptation against a co-evolving opponent rather than a fixed one.

II. RELATED WORK

A. MARL Training Paradigms

Ning and Xie [5] characterise three MARL paradigms: fully centralised (single joint policy), fully decentralised (independent learners), and Centralised Training with Decentralised Execution (CTDE). CTDE dominates because centralised execution suffers from exponential action-space growth while fully decentralised training is unstable due to non-stationarity. CTDE bridges the gap between these challenges by enabling

coordinated learning and decentralised decision-making at execution.

B. Policy-Gradient MARL

Lowe et al. [6] introduced MADDPG, the first CTDE actor-critic method, giving each agent a centralised critic. Foerster et al. [7] addressed credit assignment (determining each agent's contribution in a shared reward) with counterfactual baselines (joint action reward compared with individual action rewards). De Witt et al. [8] showed Independent PPO is competitive on hard SMAC maps, motivating Yu et al. [9] to extend PPO to multi-agent settings as MAPPO, which achieves superior performance across cooperative benchmarks including SMAC [10]. MAPPO is part of our architecture for this reason.

C. Value-Based MARL

Tan [11] proposed Independent Q-Learning which is inherently unstable due to environment non-stationarity. Sunehag et al. [12] introduced Value-Decomposition Networks (VDN). Rashid et al. [3] extended VDN with QMIX, replacing summation with a monotonic mixing network.

D. Opponent Modeling

He et al. [13] first embedded opponent observations into a DQN via a Mixture-of-Experts to implicitly capture opponent strategy without explicit prediction. Papoudakis and Albrecht [14] used variational autoencoders to infer compact latent opponent representations of opponent strategy. Liang et al. [15] proposed HERO, maintaining opponent models over high-level option predictions. Agarwal et al. [16] introduced Preference-Based Opponent Shaping (PBOS), though the approach requires white-box gradient access and is restricted to differentiable 2D settings.

E. Baselines

Jing et al. [2] (OMIS) pre-train a transformer via in-context learning (ICL) combined with Decision-Time Search (DTS) to predict and simulate opponent actions. Li et al. [1] (MARLeOM) build hierarchical opponent models via Monte-Carlo Tree Search (MCTS) over a learned world model, combined with Bayesian-weighted averaging across model levels. Chen et al. [17] address out-of-distribution generalisation with Population Based Training and an in-context transformer OM.

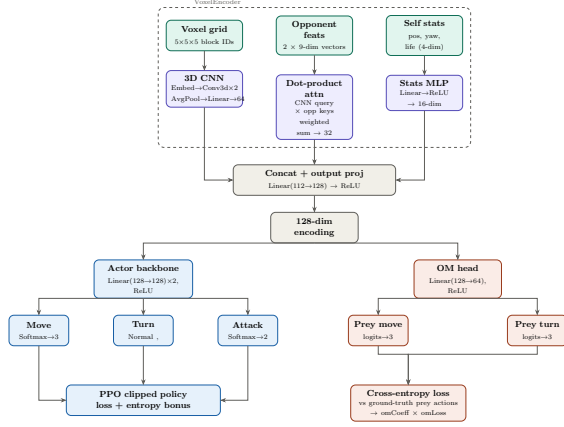


Fig. 1: MAPPO + VoxelEncoder + OM head architecture. Three branches (3D CNN, dot-product attention over opponent features, stats MLP) are fused into a 128-dim encoding shared by the actor backbone and the OM head.

III. METHODOLOGY

A. Environment

We construct a 20×20 block arena in Project Malmö [18] with 2 predators (P1, P2) and 1 prey, all learning simultaneously. Episodes terminate on a tag event, step limit ($T_{\max}=500$), or mission failure.

a) *Observation space.*: Each agent i receives a flat observation $o_i \in \mathbb{R}^{147}$ composed of:

$$o_i = \left[\underbrace{s_i}_4, \underbrace{e_1, e_2}_{2 \times 9}, \underbrace{v_i}_{125} \right] \quad (1)$$

where s_i encodes normalised position, yaw and health; each e_j encodes relative position (2), health (1), and last-action one-hot (6) of opponent j ; and $v_i \in \{0, \dots, 15\}^{125}$ is the agent's $5 \times 5 \times 5$ voxel grid. The global state for the centralised critic concatenates all agents: $g = [o_0, o_1, o_2] \in \mathbb{R}^{441}$.

b) *Action space.*: Predator actions are a tuple (move, turn, attack) where move and attack are discrete ($|\mathcal{A}_{\text{move}}|=3$, $|\mathcal{A}_{\text{attack}}|=2$) and turn is continuous $\text{turn} \in [-1, 1]$. Prey actions are (move, turn), both discrete.

c) *Reward shaping.*:

$$r_{\text{pred}}^t = \begin{cases} +10 & \text{if tag and } d_i \leq 3.5 \text{ m} \\ -0.3 & \text{otherwise (time penalty)} \end{cases} \quad (2)$$

$$r_{\text{prey}}^t = \begin{cases} -10 & \text{if tagged} \\ +0.1 & \text{otherwise (survival bonus)} \end{cases} \quad (3)$$

B. VoxelEncoder

The flat observation o_i is decomposed into its three components (Eq. 1) and processed by three parallel branches (Fig. 1):

3D CNN branch. Voxel IDs are embedded to 8-dim vectors, reshaped to $\mathbb{R}^{8 \times 5 \times 5 \times 5}$, and passed through two Conv3d layers

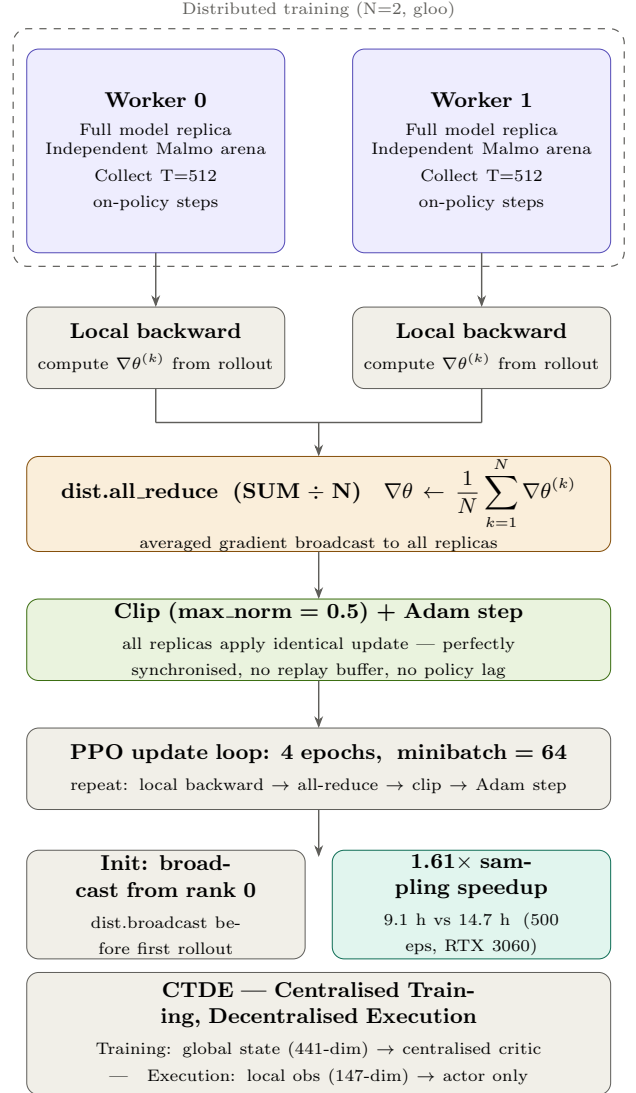


Fig. 2: Distributed training architecture with $N=2$ workers. Each worker maintains a full model replica and independent Malmö instance, collecting $T=512$ steps in parallel before gradient synchronisation via `dist.all_reduce`.

($8 \rightarrow 32 \rightarrow 64$ channels, kernel $3 \times 3 \times 3$, padding 1) followed by AdaptiveAvgPool3d(2, 2, 2) and a linear projection to 64 dimensions.

Attention branch. Opponent features $\{e_j\}$ are embedded to $\mathbb{R}^{32}$ via a linear layer. The CNN output serves as a query $q \in \mathbb{R}^{32}$ and dot-product attention with scale $1/\sqrt{32}$ produces a single weighted entity vector:

$$\alpha_j = \frac{\exp(q^\top e_j / \sqrt{d})}{\sum_k \exp(q^\top e_k / \sqrt{d})}, \quad \text{entity} = \sum_j \alpha_j e_j \quad (4)$$

Stats MLP. s_i is projected to 16 dimensions via a linear layer and ReLU.

The three outputs (64, 32, 16) are concatenated and projected:
 $\phi_i = \text{ReLU}(W[c; \text{entity}; s_i] + b) \in \mathbb{R}^{128}$.

C. Actor and OM Head

The shared predator actor backbone is a two-layer MLP (128→128) applied to ϕ_i . Three heads produce move probabilities $\pi_{\text{move}} \in \Delta^3$, turn distribution $\mathcal{N}(\mu_t, \sigma_t)$ with $\mu_t = \tanh(\cdot)$ and $\sigma_t = \exp(\text{clamp}(\cdot; -4, 1))$, and attack probabilities $\pi_{\text{attack}} \in \Delta^2$.

The OM head is a separate single-layer MLP (128→64) applied to the same encoding ϕ_i , predicting prey move and turn logits:

$$\mathcal{L}_{\text{OM}} = \text{CE}(\hat{m}_{\text{prey}}, m_{\text{prey}}) + \text{CE}(\hat{t}_{\text{prey}}, t_{\text{prey}}) \quad (5)$$

where $m_{\text{prey}}, t_{\text{prey}}$ are the ground-truth prey actions collected from the same rollout. Gradients from $\mathcal{L}_{\text{OM}}$ flow back into the VoxelEncoder, shaping the shared representation toward prey-predictive features.

D. Training Objective

For each predator we compute Generalised Advantage Estimation [19]:

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l (r_{t+l} + \gamma V(g_{t+l+1}) - V(g_{t+l})) \quad (6)$$

with $\gamma=0.99$, $\lambda=0.95$, and a shared centralised critic $V(g)$. Value targets are normalised with a running mean/variance normaliser (ValueNorm) following [9]. The full predator loss per minibatch is:

$$\mathcal{L} = \underbrace{\mathcal{L}_{\text{CLIP}}}_{\text{policy}} + \alpha_v \mathcal{L}_V - \alpha_H H[\pi] + \alpha_{\text{OM}} \mathcal{L}_{\text{OM}} \quad (7)$$

where $\mathcal{L}_{\text{CLIP}}$ is the PPO clipped surrogate [20]:

$$\mathcal{L}_{\text{CLIP}} = -\mathbb{E}_t [\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)\hat{A}_t)] \quad (8)$$

with importance ratio $r_t(\theta) = \pi_{\theta}(a_t|o_t)/\pi_{\theta_{\text{old}}}(a_t|o_t)$ and clip threshold $\varepsilon=0.15$.

The prey is trained with independent PPO using the same clipped objective but without an OM head, making it a co-evolving opponent.

Hyperparameters are summarised in Table I.

TABLE I: Hyperparameters

Parameter	Value
Discount γ	0.99
GAE λ	0.95
PPO clip ε	0.15
Entropy coeff. α_H	0.01
Value coeff. α_v	0.5
OM coeff. α_{OM}	0.5
Learning rate	3×10^{-4}
Rollout steps T	512
PPO epochs	4
Minibatch size	64
Grad clip (max norm)	0.5
Workers N	2

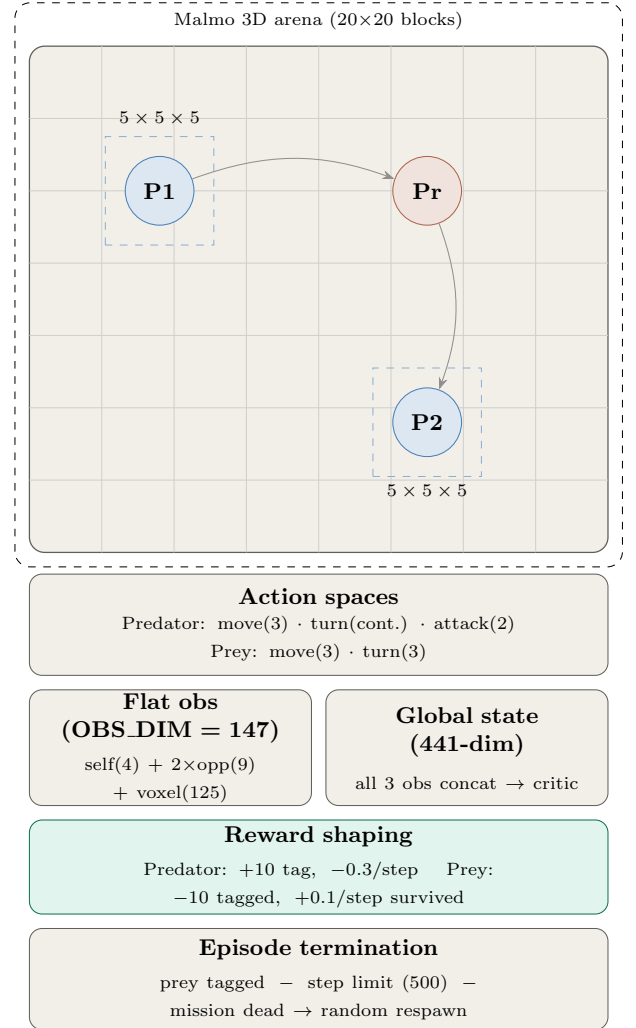


Fig. 3: Environment and distributed training setup. Two workers collect rollouts in parallel from independent Malmo arenas, compute local gradients, synchronise via `dist.all_reduce`, and apply an identical Adam step.

E. Distributed Rollout Collection

Our distributed approach runs N identical workers having a separate environment and full copy of the current policy. The environment is sampled by all workers in parallel. Gradients are computed locally and then synchronised via all-reduce, so every worker takes the same gradient step. We note that this causes synchronisation lag which is unavoidable.

Following Liu et al. [21], each of $N=2$ workers maintains a full model replica and an independent Malmo arena (Fig. 2). Workers collect $T=512$ on-policy steps in parallel, then synchronise via `dist.all_reduce` (gloo backend, file-based rendezvous):

$$\nabla\theta \leftarrow \frac{1}{N} \sum_{k=1}^N \nabla\theta^{(k)} \quad (9)$$

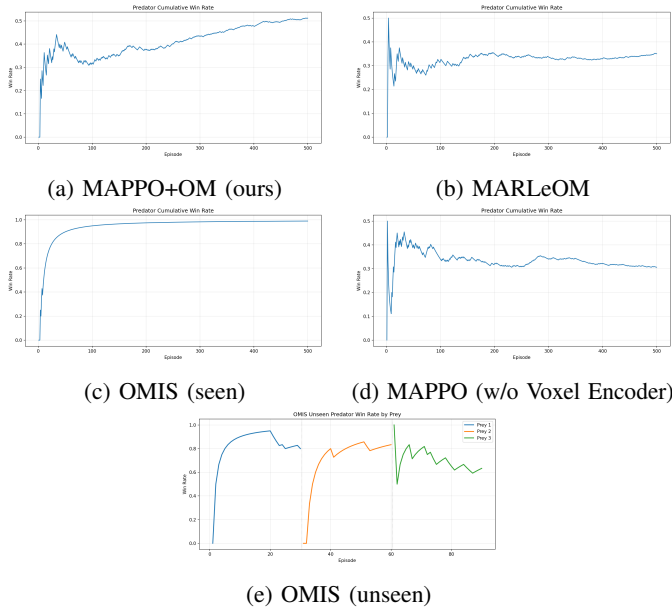


Fig. 4: Win rates across all methods over 500 episodes.

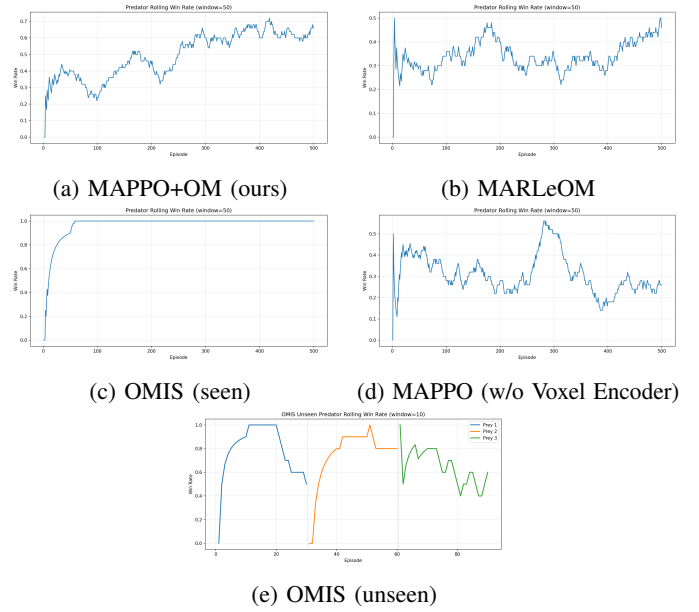


Fig. 5: Predator rolling win rate (50-episode window).

All replicas apply the same averaged gradient. Initial weights are broadcast from rank 0 before the first rollout (`dist.broadcast`); see Fig. 3 for the full setup.

IV. RESULTS

A. Baselines

a) *OMIS* [2].: We implement single-step critic-guided imitator heads in place of MCTS-backed DTS, as obtaining the environment model is non-trivial. Shared predator networks are trained against pre-trained, fixed prey policies to keep the computation manageable under a transformer architecture.

b) *MARLeOM* [1].: We omit the world model and limit Opponent Model (OM) depth to two levels (L0/L1). L0 is trained to clone the behaviour of the opponent. Level-1 best response uses one-step critic guidance instead of full rollout planning.

B. Win Rate

Figures 4 and 5 show cumulative and rolling win rates. Results are summarised in Table II.

TABLE II: Predator win rates after 500 episodes

Method	Rolling WR (final)	Cumulative WR
MAPPO + OM (ours)	~ 0.65	~ 0.51
MARLeOM	~ 0.50	~ 0.35
OMIS (seen prey)	~ 1.00	~ 0.99
MAPPO (w/o Voxel Encoding)	~ 0.26	~ 0.31
OMIS (unseen prey)	~ 0.63	~ 0.77

MARLeOM underperforms because its MCTS-based planning assumes a learned world model and discrete 2D state transitions; both assumptions break in our setting. OMIS achieves near-perfect win rates on seen prey policies since it pre-trains directly on those trajectories, but it sees the fixed

prey policy during training — an advantage that does not hold against a co-evolving opponent. Additionally, against 3 unseen opponents, it gives similar rolling win rates. The pre-training steps and the policies being scripted makes the strategy obsolete against co-learning opponents. We also train MAPPO without Voxel Encoding i.e. no CNN encoder to preprocess Voxel data. It is passed directly into the OM Head. The results show its importance with rolling win rate falling twice as lower due to instability of the training.

C. Distributed Speedup

Parallel rollout collection with $N=2$ workers reduces wall-clock time from 14.67 h to 9.13 h ($1.61\times$ speedup) for 500 episodes on an RTX 3060. The sub-linear scaling relative to $N=2$ is attributable to all-reduce synchronisation overhead, Malmo episode-reset latency, and the per-step environment sleep of 0.1 s.

V. CONCLUSION

We presented MAPPO extended with a VoxelEncoder and jointly-trained OM head for a 3D predator-prey arena in Project Malmo. Against a co-learning prey the method achieves a cumulative win rate of 0.51 and rolling win rate of 0.65, outperforming adapted MARLeOM and OMIS implementations. Distributed rollout collection yields a $1.61\times$ sampling speedup. Both baselines require architectural changes to operate in 3D continuous-observation settings, which points to spatially-aware encoders as a necessary component for OM in 3D MARL.

Single-seed evaluation and a 500-episode training budget were the primary constraints, both imposed by compute limits on the RTX 3060 (8+ hours per run). Ablations on α_{OM} and scaling beyond $N=2$ workers are left for future work.

REFERENCES

- [1] S. Li, X. Su, and B. Shi, “A multi-agent reinforcement learning algorithm embedded with opponent modeling,” in *Proc. 27th European Conference on Artificial Intelligence (ECAI)*. IOS Press, 2024.
- [2] Y. Jing, B. Liu, K. Li, Y. Zang, H. Fu, Q. Fu, J. Xing, and J. Cheng, “Opponent modeling with in-context search,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.
- [3] T. Rashid, M. Samvelyan, C. Schroeder de Witt, G. Farquhar, J. Foerster, and S. Whiteson, “QMIX: Monotonic value function factorisation for deep multi-agent reinforcement learning,” in *Proc. 35th International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 80, 2018, pp. 4295–4304.
- [4] Z. Lin, J. Li, J. Shi, D. Ye, Q. Fu, and W. Yang, “JueWu-MC: Playing minecraft with sample-efficient hierarchical reinforcement learning,” in *Proc. 31st International Joint Conference on Artificial Intelligence (IJCAI)*, 2022.
- [5] Z. Ning and L. Xie, “A survey on multi-agent reinforcement learning and its application,” *Journal of Automation and Intelligence*, 2024.
- [6] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, “Multi-agent actor-critic for mixed cooperative-competitive environments,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [7] J. N. Foerster, G. Farquhar, T. Afouras, N. Nardelli, and S. Whiteson, “Counterfactual multi-agent policy gradients,” in *Proc. 32nd AAAI Conference on Artificial Intelligence (AAAI)*, 2018, pp. 2974–2982.
- [8] C. S. de Witt, T. Gupta, D. Makoviichuk, V. Makoviychuk, P. H. S. Torr, M. Sun, and S. Whiteson, “Is independent learning all you need in the StarCraft multi-agent challenge?” *arXiv preprint arXiv:2011.09533*, 2020.
- [9] C. Yu, A. Velu, E. Vinitzky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, “The surprising effectiveness of PPO in cooperative multi-agent games,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 24 611–24 624.
- [10] M. Samvelyan, T. Rashid, C. Schroeder de Witt, G. Farquhar, N. Nardelli, T. G. J. Rudner, C.-M. Hung, P. H. S. Torr, J. Foerster, and S. Whiteson, “The StarCraft multi-agent challenge,” in *Proc. 18th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2019.
- [11] M. Tan, “Multi-agent reinforcement learning: Independent vs. cooperative agents,” in *Proc. 10th International Conference on Machine Learning (ICML)*. Morgan Kaufmann, 1993, pp. 330–337.
- [12] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls, and T. Graepel, “Value-decomposition networks for cooperative multi-agent learning based on team reward,” in *Proc. 17th International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2018, pp. 2085–2087.
- [13] H. He, J. Boyd-Graber, K. Kwok, and H. Daumé III, “Opponent modeling in deep reinforcement learning,” in *Proc. 33rd International Conference on Machine Learning (ICML)*, ser. PMLR, vol. 48, 2016, pp. 1804–1813.
- [14] G. Papoudakis and S. V. Albrecht, “Variational autoencoders for opponent modeling in multi-agent systems,” in *AAAI 2020 Workshop on Reinforcement Learning in Games*, 2020.
- [15] Z. Liang, J. Cao, S. Jiang, D. Saxena, and H. Xu, “Hierarchical reinforcement learning with opponent modeling for distributed multi-agent cooperation,” in *Proc. IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2022.
- [16] A. Agarwal, J. Stoyanovich, and J. Foerster, “Preference-based opponent shaping in differentiable games,” *Machine Learning*, 2025.
- [17] J. Chen, L. Wu, C. Wen, and J. Wang, “An open-ended learning framework for opponent modeling,” in *Proc. 39th AAAI Conference on Artificial Intelligence (AAAI)*, 2025.
- [18] M. Johnson, K. Hofmann, T. Hutton, and D. Bignell, “The malmo platform for artificial intelligence experimentation,” in *Proc. 25th International Joint Conference on Artificial Intelligence (IJCAI)*. AAAI Press, 2016, pp. 4246–4247.
- [19] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, “High-dimensional continuous control using generalized advantage estimation,” 2015.
- [20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [21] Z. Liu, X. Xu, P. Qiao, and D. Li, “Acceleration for deep reinforcement learning using parallel and distributed computing: A survey,” *ACM Computing Surveys*, vol. 57, no. 4, pp. 1–35, 2024.